

Milestone 3 — Simulation Engine

Design Checkpoint & Code Review

Monte Carlo Options Pricing System

August 11, 2026

Contents

1	Milestone Objective	2
2	Architecture	2
2.1	File Layout	2
2.2	Data Flow	2
3	Locked Design Decisions	2
4	Model Mathematics	2
4.1	GBM — Exact Solution	2
4.2	Merton Jump-Diffusion	3
4.3	Heston Stochastic Volatility	3
4.3.1	QE Variance Step	3
4.3.2	Log-Price Discretization	4
5	Variance Reduction	4
5.1	Antithetic Variates	4
5.2	Stratified Sampling	4
5.3	Measured Variance Reduction	4
6	Implementation Judgment Calls	4
6.1	<code>mu</code> on <code>HestonSimulator</code>	4
6.2	<code>simulate_with_variance()</code>	5
7	M2 Bug Fix: JD Calibrator <code>mu</code> Convention	5
7.1	The Problem	5
7.2	The Fix	5
7.3	Round-Trip Verification	5
7.4	Note on Sign	5
8	Test Coverage	5
8.1	Required Tests	6
8.2	Extended Verification	6
9	Performance	6
10	Cumulative Project State	6
10.1	Next: M4 — Options Pricing Module	6

1 Milestone Objective

Generate Monte Carlo price paths under three stochastic models—GBM, Merton jump-diffusion, and Heston stochastic volatility—with full vectorization, reproducible seeding, and pluggable variance reduction. The simulation engine consumes calibrated parameters from M2 and produces path arrays that M4’s pricing module will use to price options.

2 Architecture

2.1 File Layout

File	Responsibility
base.py	SimulatorBase ABC, input validation, log-space path builder
gbm.py	Exact GBM solution
jump_diffusion.py	Vectorized Poisson, compensated drift
heston.py	Andersen QE scheme, private helpers
variance_reduction.py	generate_samples() — antithetic & stratified
utils.py	resolve_n_steps, make_rng, path statistics

2.2 Data Flow

Calibrated Params (M2)

```

|
v
SimulatorBase.simulate(S0, params, T, dt, n_sims, seed, var_red)
|
+--> generate_samples(rng, shape, method)    [variance_reduction.py]
|      |
|      +--> plain / antithetic / stratified normals
|
+--> model-specific log-increment computation
|
+--> _paths_from_log_increments(S0, log_increments)
|
v
np.ndarray of shape (n_simulations, n_steps + 1)

```

3 Locked Design Decisions

4 Model Mathematics

4.1 GBM — Exact Solution

The GBM SDE and its exact solution at discrete steps:

$$dS = \mu S dt + \sigma S dW \tag{1}$$

$$S_{t+\Delta t} = S_t \cdot \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t} Z\right], \quad Z \sim \mathcal{N}(0, 1) \tag{2}$$

This is exact at the step dates for any Δt —no Euler discretization error. The $-\sigma^2/2$ term is the Itô correction ensuring $E[S_T] = S_0 e^{\mu T}$.

#	Decision	Rationale
1	Stateless simulators	<code>simulate()</code> returns paths, stores nothing on <code>self</code> . Decouples path generation from path analysis.
2	<code>np.random.default_rng(seed)</code>	Isolated <code>Generator</code> per call. No global state, no cross-contamination. MLflow reproducibility guaranteed.
3	Derive <code>n_steps</code> inside	<code>n_steps = round(T/dt)</code> . Caller thinks in years and step size, not step count.
4	Single-jump approximation	Vectorized Poisson; valid when $\lambda_j \cdot dt < 0.1$. Compensated drift preserves $E[S_T] = S_0 e^{\mu T}$.
5	Heston QE scheme	Andersen (2008), $\psi_{\text{crit}} = 1.5$. Private methods in <code>heston.py</code> . Loop over steps, vectorized across sims.
6	Variance reduction as parameter	<code>variance_reduction</code> on <code>simulate()</code> , delegates to <code>generate_samples()</code> for Z generation.

4.2 Merton Jump-Diffusion

Extends GBM with compound Poisson jumps:

$$\frac{dS}{S} = (\mu - \lambda k) dt + \sigma dW + J dN \quad (3)$$

$$N \sim \text{Poisson}(\lambda \Delta t), \quad J \sim \mathcal{N}(\mu_j, \sigma_j^2) \quad (4)$$

where $k = e^{\mu_j + \sigma_j^2/2} - 1$ is the expected proportional jump size. The compensator $-\lambda k$ in the drift ensures jumps add risk but not expected return.

Vectorized implementation. Jump counts and sizes are drawn for all (sim, step) pairs at once. A boolean mask `1[N > 0]` zeroes out steps with no jump. The single-jump approximation is validated by requiring $\lambda \cdot \Delta t < 0.1$.

4.3 Heston Stochastic Volatility

Price and variance follow coupled SDEs:

$$dS = \mu S dt + \sqrt{v} S dW_S \quad (5)$$

$$dv = \kappa(\theta - v) dt + \xi \sqrt{v} dW_v \quad (6)$$

$$\text{Corr}(dW_S, dW_v) = \rho \quad (7)$$

Feller condition: $2\kappa\theta > \xi^2$ ensures variance stays positive in continuous time. With discrete steps, a naive Euler scheme can still produce $v < 0$.

4.3.1 QE Variance Step

Given v_t , the conditional distribution of $v_{t+\Delta t}$ is non-central χ^2 . Its first two moments:

$$m = \theta + (v_t - \theta) e^{-\kappa \Delta t} \quad (8)$$

$$s^2 = \frac{v_t \xi^2 e^{-\kappa \Delta t}}{\kappa} (1 - e^{-\kappa \Delta t}) + \frac{\theta \xi^2}{2\kappa} (1 - e^{-\kappa \Delta t})^2 \quad (9)$$

The shape ratio $\psi = s^2/m^2$ determines the proxy distribution:

- $\psi \leq 1.5$ — **Quadratic:** $v_{t+\Delta t} = a(b + Z)^2$ where a, b are moment-matched. Non-negative by construction (squared).

- $\psi > 1.5$ — **Exponential**: point mass p at zero plus exponential tail. Non-negative by construction.

Both branches consume the same uniform u (inverted to a normal for the quadratic branch via Φ^{-1}), preserving per-path reproducibility when the branch switches.

4.3.2 Log-Price Discretization

The log-price increment uses Andersen’s K_0 – K_4 constants with central weighting $\gamma_1 = \gamma_2 = 0.5$:

$$\ln \frac{S_{t+\Delta t}}{S_t} = \mu \Delta t + K_0 + K_1 v_t + K_2 v_{t+\Delta t} + \sqrt{K_3 v_t + K_4 v_{t+\Delta t}} Z \quad (10)$$

where Z is independent of the variance innovations. The constants fold the correlation structure (ρ) into deterministic coefficients of v_t and $v_{t+\Delta t}$.

Verification: with $\rho = 0$ and $\xi \rightarrow 0$ (constant variance), $K_1 v + K_2 v = -\frac{1}{2}v \Delta t$ and $\sqrt{(K_3 + K_4)v} = \sqrt{v \Delta t}$, recovering the exact GBM formula. Confirmed by repricing a European call against Black-Scholes within Monte Carlo error.

5 Variance Reduction

Both methods operate on the standard normal draws Z via a single entry point `generate_samples(rng, shape, method)`, leaving path-evolution code untouched.

5.1 Antithetic Variates

For n simulations, draw $\lceil n/2 \rceil$ base normals $Z_1, \dots, Z_{\lceil n/2 \rceil}$ and pair each with $-Z_i$. Path functionals monotone in the driving noise get a negatively correlated partner, reducing estimator variance.

5.2 Stratified Sampling

Divide $[0, 1]$ into n equal-probability strata. Draw one uniform per stratum per time step, apply Φ^{-1} to obtain normals. Strata are shuffled independently per column to keep time steps independent.

5.3 Measured Variance Reduction

Over 60 independent runs of the $E[S_T]$ estimator (500 paths each):

Method	Variance Ratio vs. Naive
Antithetic	22× lower
Stratified	81× lower (single-step; narrows with more steps)

6 Implementation Judgment Calls

6.1 mu on HestonSimulator

`HestonParams` carries no drift—the calibrator fits the variance term structure only. `simulate()` accepts `mu: float = 0.0` as an additional keyword argument, defaulting to risk-neutral dynamics at a zero rate. M4 pricing will pass $r - q$.

6.2 `simulate_with_variance()`

`HestonSimulator` exposes a second public method returning both price and variance paths. `simulate()` delegates to it and drops the variance array. Needed for non-negativity testing and future variance derivatives.

7 M2 Bug Fix: JD Calibrator μ Convention

7.1 The Problem

The MLE in the jump-diffusion calibrator fits the log-space diffusion drift:

$$\alpha = \mu - \lambda k - \frac{1}{2}\sigma^2 \quad (11)$$

The original code stored $\alpha + \frac{1}{2}\sigma^2$ as `mu`, missing the λk term. Since the simulator subtracts λk internally as the compensator, the correction was double-applied, understating drift by λk per year.

7.2 The Fix

$$\text{mu} = \alpha + \frac{1}{2}\sigma^2 + \lambda k \quad \text{where} \quad k = e^{\mu_j + \sigma_j^2/2} - 1 \quad (12)$$

Both calibrators now speak the same convention: `mu` is the expected return of the price process, $E[S_T] = S_0 e^{\mu T}$.

7.3 Round-Trip Verification

Synthetic data with $\lambda = 2$, $\mu_j = -0.05$ ($\lambda k \approx -0.095$, five times the Itô term):

Formula	Simulated Drift	Source Drift
$\alpha + \sigma^2/2$ only	+0.1796	+0.0202
$\alpha + \sigma^2/2 + \lambda k$	+0.0218	+0.0202

7.4 Note on Sign

When jumps have negative mean ($\mu_j < 0$), the compensator λk is negative, so `mu` can be *below* the log-return mean. This is correct—downward jumps reduce the expected price level, so the price-level drift must be lower to compensate—but reads as surprising if one expects the GBM relationship $\mu = \bar{r}_{\log} + \sigma^2/2$ to always hold.

8 Test Coverage

96 tests passing (72 new for M3, 4 new for the M2 fix, 20 from M2).

Test	What It Verifies
GBM mean path convergence	$E[\bar{S}_T] \rightarrow S_0 e^{\mu T}$ as $N \rightarrow \infty$
GBM terminal KS test	Terminal distribution is log-normal (Kolmogorov-Smirnov)
JD with $\lambda = 0$	Equivalent to GBM output
Heston non-negativity	$v_t \geq 0$ for all paths and steps, under Feller-violating params
Heston vacuity guard	Variance actually reaches zero under the violating params
Antithetic variance reduction	Estimator variance lower than naive over 60 runs
Stratified variance reduction	KS test confirms better distributional coverage
Seed reproducibility	Same seed $\Rightarrow$ identical arrays

Test	What It Verifies
Heston $\xi \rightarrow 0$ BS reprice	With frozen variance, Heston reprices a European call to within MC error of Black-Scholes
JD mu round-trip (jumpy data)	Simulate with known μ , calibrate back, recovered μ matches within tolerance ($\lambda = 2$, $\mu_j = -0.05$)
JD round-trip fails without λk	Stripping λk from recovered params produces > 0.05 error
JD mu is price-level drift	Cross-checks μ against sample mean simple return

8.1 Required Tests

8.2 Extended Verification

9 Performance

10 Cumulative Project State

10.1 Next: M4 — Options Pricing Module

The simulation engine produces path arrays; M4 consumes them to price derivatives. Key components:

- Payoff functions (European, Asian, barrier, lookback)
- MC pricer: simulate $\rightarrow$ compute payoffs $\rightarrow$ discount $\rightarrow$ statistics
- Black-Scholes closed-form for benchmarking
- Greeks via bump-and-revalue (finite differences with common random numbers)

Model	Target	Notes
GBM ($10k \times 252$)	< 0.5s	Fully vectorized, no Python loops
Heston ($10k \times 252$)	< 1.0s	Loop over 252 steps, vectorized across sims

Milestone	Status	Tests
M1 — Data Ingestion	Complete	20
M2 — Calibration Engine	Complete (JD fix applied)	24
M3 — Simulation Engine	Complete	72
Total		96